

Report for Mandatory Assignment 3 (Chitty-Chat)

Group name: JAA

Members(abch,asjo,jnov)

28/10-2024

- **Describe your system architecture - do you have a server-client architecture, peer-to-peer, or something else?**
 - The system uses a *server-client architecture*, where all clients communicate to each other through the server. When a client joins the server, the server will broadcast all messages that has been previously sent to the newly joined client. When a client leaves or joins the service, the server will broadcast the relevant information to all participating clients. Clients can send as many messags as they like, and the server will broadcasts these messages to all participating clients,
- **Describe what RPC methods are implemented, of what type, and what messages types are used for communication**
 - The service has three methods: **Broadcast**, **Publish**, and **Leave**.
 - The messages types used for comminication are:
 - **Participant**{string Name}
 - **Message**{string message, Participant par, uint32 LamportTime}
 - **Empty**{}

Broadcast requests a *Participant* and will receive a stream of *messages*. This RPC method is invoked when a client has successfully established a connection to the service. The method will use the clients name, and create and store a *join* message on the server, which is then broadcasted to all participating clients. The method will also store any message that a client publishes, on the server.

Publish requests a *message* and receives *empty*. This RPC method allows a client to publish a message. All published messages are stored on the server.

Leave requests *Participant* and receives *empty*. This RPC method allows a client to leave the chat. When the method is invoked it will create and store a *leave* message on the server, which then will be broadcasted to all participating clients.

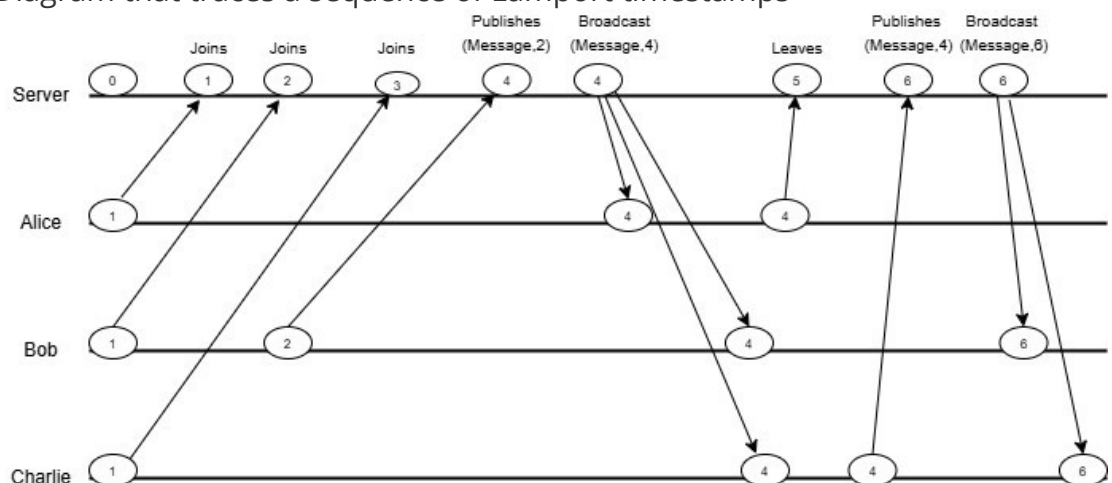
- **Describe how you have implemented the calculation of the Lamport timestamps**

- Server and Clients each have a Uint32 LamportTime stamp which starts at 0 for both. Whenever a RPC method is called the ServerLamport is incremented by 1.

When a client receives a *message* the client compares its own Lamport timestamp with the message's timestamp. If the message's timestamp is ahead, the client updates its own timestamp to match it and increments it. If the received timestamp is not ahead, the client just increments its own timestamp.

- **Provide a diagram, that traces a sequence of RPC calls together with the Lamport timestamps, that corresponds to a chosen sequence of interactions: Client X joins, Client X Publishes, ..., Client X leaves. Include documentation (system logs) in your appendix**

- Diagram that traces a sequence of Lamport timestamps



-System logs from the sequence above:

Server:

```
Listening on: 0.0.0.0:50051
-> Participant Alice joined Chitty-Chat at Lamport time 1
-> Participant Bob joined Chitty-Chat at Lamport time 2
-> Participant Charlie joined Chitty-Chat at Lamport time 3
-> Participant Alice Left Chitty-Chat at Lamport time 5
```

```
□
```

Alice(first client):

```
15 C:\Users\jager\OneDrive\Documents\chitty-chat\chitty-chat\chitty_chat.py 80 Pan 1 name Alice
Welcome to Chitty-chat Alice! To send a message type it in below and hit enter! To exit the chat type in 'Quit', and hit enter.
Following messages have already been sent:
(1) You: Joined Chitty-Chat
(2) Bob: Joined Chitty-Chat
(3) Charlie: Joined Chitty-Chat
(4) Bob: Hello this is a message from Bob!
Quit
(5) You: Left Chitty-Chat
```

```
86 C:\Users\jager\OneDrive\Documents\chitty-chat\chitty-chat\chitty_chat.py 86 Pan 1 name Alice
```

Bob(second client):

```
Welcome to Chitty-chat Bob! To send a message type it in below and hit enter! To exit the chat type in 'Quit', and hit enter.
Following messages have already been sent:
(1) Alice: Joined Chitty-Chat
(2) You: Joined Chitty-Chat
(3) Charlie: Joined Chitty-Chat
Hello this is a message from Bob!
(4) You: Hello this is a message from Bob!
(5) Alice: Left Chitty-Chat
(6) Charlie: Hello this is a message from Charlie!
```

```
□
```

Charlie(third client):

```
Welcome to Chitty-chat Charlie! To send a message type it in below and hit enter! To exit the chat type in 'Quit', and hit enter.
Following messages have already been sent:
(1) Alice: Joined Chitty-Chat
(2) Bob: Joined Chitty-Chat
(3) You: Joined Chitty-Chat
(4) Bob: Hello this is a message from Bob!
(5) Alice: Left Chitty-Chat
Hello this is a message from Charlie!
(6) You: Hello this is a message from Charlie!
```

```
□
```

Link for git repository:

https://github.com/Asger124/Distributed/tree/Asger_tcp/Chitty_Chat